3C · CLAUDE CODE COACH · SECURITY ONE-PAGER

# Where your code goes: nowhere it didn't already.

The under-claiming summary — what L1 produces, and what your org always owns.

## Data flow // trust boundary

The developer workstation is the trust boundary. 3C adds no path off the workstation for source code, prompt content, or credentials, under any maturity level.

At L2+, the only thing 3C optionally emits is anonymised metadata — counts, pass/fail, durations — to a self-hostable backend. It does not change how your AI assistant already operates with its own model provider; that remains governed by your existing agreement with that vendor.

## Security posture // by design

- **OWASP-aligned generation** — security rule sets aligned to the OWASP Top 10 applied at the point of generation (VP-01-F04), before review.
- **Secret-scanning** — every file-writing skill scans for secrets before the write lands.
- **Local storage** — config + event state on the workstation (SQLite + JSONL); no code or prompt content off-box.
- **Offline resilient** — the workflow never blocks on a backend; no always-on egress.

## What 3C does NOT do // honest

3C is a developer-productivity and standards-alignment framework, **not a compliance product**. It does **not** certify any standard or produce a regulatory evidence pack — that is a separate, future business case.

3C is not a SAST replacement: it shifts secure-coding defaults left (OWASP Top 10 at the point of generation), it does not absolve your existing security review, scanning, and reviewer training.

## Evaluate it as OSS // not a vendor

- **Licence:** MIT — read it in the repo `LICENSE`.
- **SBOM:** `npm sbom --sbom-format cyclonedx` after install captures 3C + transitive deps; git-installed, so recorded with `git+https` provenance.
- **Source of truth:** every claim is grounded in the public repo `kgn-git/3C` — evaluators can open a GitHub discussion to walk the requirements baseline.